

BAB VI

PERFORMANCE METRICS – METRIK EVALUASI KUALITAS PENJELASAN

Bab ini akan membahas metrik yang digunakan untuk mengevaluasi kualitas penjelasan yang dihasilkan oleh GraphXAI, seperti Graph Explanation Accuracy (GEA), Graph Explanation Faithfulness (GEF), Graph Explanation Stability (GES), dan Graph Explanation Fairness (GECF, GEGF). Selain itu, bab ini juga akan menyajikan hasil uji coba dari penerapan GEA dan GEF.

6.1 Performance Metrics dalam GraphXAI

Metrik evaluasi explainability sangat penting untuk menilai apakah penjelasan yang diberikan oleh model GNN sesuai dengan pengambilan keputusan asli yang dilakukan oleh model tersebut. Tanpa metrik yang jelas dan terstandarisasi, sangat sulit untuk memastikan apakah penjelasan yang diberikan benar-benar relevan dan dapat dipercaya. Evaluasi ini juga penting untuk mengidentifikasi potensi bias atau kesalahan dalam model yang mungkin tidak terlihat pada sekadar pengujian akurasi model. Seiring dengan munculnya berbagai metode explainability seperti GradExplainer, GNNExplainer, PGMEExplainer dan GraphLIME, evaluasi dengan metrik yang objektif dan terukur memungkinkan kita untuk membandingkan kinerja dari setiap metode dalam menjelaskan keputusan model. Tanpa evaluasi yang memadai, sangat sulit untuk mengetahui apakah metode explainability tertentu benar-benar memberikan hasil yang valid tentang bagaimana model memproses graf dan mengambil keputusan.

Dalam konteks ini, GraphXAI menawarkan berbagai metrik untuk mengevaluasi kualitas penjelasan yang diberikan oleh berbagai metode explainability pada GNN. Metrik-metrik ini memainkan peran yang sangat penting dalam memastikan bahwa penjelasan yang diberikan oleh model tidak hanya relevan dan logis, tetapi juga mencerminkan proses pengambilan keputusan yang

sebenarnya dalam model GNN. Pada bab ini, akan mendalami empat metrik evaluasi utama yang digunakan untuk menilai kualitas penjelasan pada GNN, yaitu:

1. Graph Explanation Accuracy (GEA): Metrik ini mengukur seberapa baik penjelasan yang diberikan mencerminkan elemen-elemen graf yang benar-benar penting bagi prediksi model. Dalam pengujian ini, nilai GEA yang lebih tinggi menunjukkan bahwa penjelasan yang diberikan oleh model sesuai dengan fitur atau hubungan antar-node yang relevan dan berpengaruh terhadap keputusan model.
2. Graph Explanation Faithfulness (GEF): Metrik ini mengukur seberapa baik penjelasan yang diberikan sesuai dengan cara kerja model yang sebenarnya. GEF yang tinggi menandakan bahwa penjelasan tidak hanya logis tetapi juga setia terhadap cara model bekerja dalam membuat keputusan. Metrik ini penting untuk memastikan bahwa penjelasan yang diberikan memang relevan dengan keputusan model, dan tidak menyimpang dari pengambilan keputusan yang asli.
3. Graph Explanation Stability (GES): GES mengukur stabilitas penjelasan model jika data mengalami perubahan kecil. Penjelasan yang stabil menunjukkan bahwa meskipun ada variasi dalam data input, penjelasan tetap konsisten. Metrik ini penting untuk menilai apakah penjelasan yang diberikan oleh model dapat bertahan terhadap gangguan kecil atau noise dalam data.
4. Graph Explanation Fairness (GECF & GEGF): GECF dan GEGF mengukur apakah penjelasan model cenderung bias terhadap node atau fitur tertentu yang tidak relevan. Metrik ini mengidentifikasi ketidakseimbangan atau bias dalam penjelasan yang diberikan oleh model, yang penting untuk memastikan bahwa model tidak membuat keputusan berdasarkan fitur yang tidak relevan atau diskriminatif. Kedua metrik ini juga menilai apakah penjelasan yang diberikan oleh model adil bagi semua elemen dalam graf, tanpa memberi bobot yang berlebihan pada fitur atau node tertentu yang tidak semestinya.

Dalam penelitian ini, akan dilakukan pengukuran kinerja Graph Explanation Accuracy (GEA) dan Graph Explanation Faithfulness (GEF) untuk

membandingkan performa dari empat metode explainer yaitu GradExplainer, GNNExplainer, PGMEExplainer, dan GraphLIME. Evaluasi ini tidak hanya memberikan gambaran mengenai kualitas penjelasan, tetapi juga menunjukkan apakah penjelasan tersebut sesuai dengan keputusan model, serta menilai sejauh mana model bisa menjelaskan pengambilan keputusan dengan cara yang transparan.

Pada subbab berikutnya, akan membahas secara lebih rinci tentang tiap metriknya dan implementasi evaluasi dengan menggunakan kode yang telah disiapkan untuk mengukur GEA dan GEF, serta hasil evaluasi dari masing-masing explainer.

6.1.1 Graph Explanation Accuracy (GEA)

Metrik ini mengukur seberapa baik penjelasan graf yang dihasilkan oleh model berada dalam kesepakatan dengan penjelasan ground-truth. Ini dilakukan dengan membandingkan penjelasan prediksi dengan penjelasan ground-truth menggunakan indeks Jaccard, sehingga didapatkan rumus seperti di bawah ini.

$$GEA(\zeta, M^p) = \max JAC(M^g, M^p) \forall M^g \in \zeta \quad \dots\dots\dots(6.1)$$

Dalam membandingkan penjelasan prediksi (M^p) dengan penjelasan ground-truth (M^g), setiap node, fitur node, atau edge dalam graf dapat dianggap penting atau tidak penting dalam prediksi model, diwakili dengan nilai $\{0, 1\}$, di mana '0' berarti atribut tersebut tidak penting dan '1' berarti atribut tersebut penting.

Indeks Jaccard dihitung berdasarkan True Positives (TP), False Positives (FP), dan False Negatives (FN). TP merujuk pada kasus di mana atribut yang dianggap penting oleh model benar-benar merupakan atribut yang penting menurut penjelasan *ground truth*. FP adalah atribut yang dianggap penting oleh model tetapi sebenarnya tidak penting, sedangkan FN adalah atribut yang penting menurut *ground truth* tetapi tidak dianggap penting oleh model.

Karena banyak graf, baik yang sintetis maupun yang berasal dari dunia nyata, memiliki berbagai penjelasan *ground truth* yang sah, maka akurasi harus mempertimbangkan berbagai penjelasan yang mungkin ada untuk setiap prediksi tertentu. Oleh karena itu, ζ didefinisikan sebagai himpunan dari semua kemungkinan penjelasan *ground truth*, di mana $|\zeta|=1$ untuk graf yang hanya memiliki satu penjelasan yang unik. Maka, GEA dihitung dengan mencari kesamaan antara penjelasan yang diprediksi dengan *ground truth*, menggunakan indeks Jaccard ini.

Peningkatan nilai GEA menunjukkan bahwa penjelasan yang diprediksi semakin cocok dengan penjelasan yang benar, yang berarti penjelasan tersebut lebih baik dan lebih dapat dipercaya dalam menggambarkan alasan di balik keputusan model. Sebaliknya, nilai GEA yang rendah mengindikasikan bahwa penjelasan yang diberikan model tidak akurat dan kurang relevan dengan pengambilan keputusan yang sesungguhnya.

6.1.2 Graph Explanation Faithfulness (GEF)

Graph Explanation Faithfulness (GEF) mengukur sejauh mana penjelasan yang diberikan oleh model GNN mencerminkan proses pengambilan keputusan asli dari model tersebut. Dalam konteks ini, GEF bertujuan untuk memastikan bahwa penjelasan yang dihasilkan benar-benar menggambarkan bagaimana model GNN membuat prediksi, bukan sekadar menunjukkan informasi yang relevan secara sembarangan. Untuk mengukur GEF, menggunakan Kullback-Leibler (KL) divergence, yang mengukur perbedaan antara dua distribusi probabilitas.

Secara lebih rinci, GEF mengukur perbedaan antara probabilitas prediksi yang dihasilkan oleh model GNN berdasarkan subgraf asli dan subgraf yang dihasilkan penjelasan. Proses ini dimulai dengan menghasilkan dua set prediksi, satu berdasarkan graf asli dan satu lagi berdasarkan subgraf yang hanya menyimpan fitur-fitur teratas yang diidentifikasi oleh penjelasan. Selanjutnya, akan dihitung KL divergence antara dua distribusi probabilitas tersebut. KL divergence memberikan ukuran seberapa besar perbedaan antara dua distribusi probabilitas. Nilai KL divergence yang tinggi menunjukkan perbedaan yang besar antara kedua distribusi,

yang berarti penjelasan tidak terlalu mencerminkan cara kerja model GNN. Sebaliknya, nilai yang rendah menunjukkan bahwa penjelasan cukup setia terhadap prediksi model. Formulasi dari GEF adalah sebagai berikut:

$$GEF(f, S_w, S_u) = 1 - \exp\left(-KL(f(S_w) \parallel f(S_u))\right) \dots\dots\dots(6.2)$$

Di mana:

- $f(S_w)$ adalah probabilitas prediksi yang diperoleh dari model GNN berdasarkan graf yang dimodifikasi dengan fitur penting yang dipilih oleh penjelasan.
- $f(S_u)$ adalah probabilitas prediksi dari graf asli.
- KL divergence mengukur perbedaan antara distribusi prediksi $f(S_w)$ dan $f(S_u)$.

Jika nilai GEF mendekati 1, ini berarti penjelasan sangat tidak setia terhadap prediksi model, karena ada perbedaan yang besar antara prediksi dengan subgraf yang dihasilkan penjelasan dan prediksi dengan graf asli. Sebaliknya, nilai GEF yang lebih rendah menunjukkan bahwa penjelasan lebih setia terhadap cara model bekerja. Metrik ini penting karena penjelasan yang tidak setia dapat menyebabkan interpretasi yang salah terhadap bagaimana model GNN membuat prediksi, yang berisiko menyebabkan kesalahan dalam pengambilan keputusan berbasis AI.

6.1.3 Graph Explanation Stability (GES)

Graph Explanation Stability (GES) mengukur sejauh mana penjelasan yang diberikan oleh model tetap konsisten atau stabil ketika dilakukan perubahan kecil (perturbasi) pada data input, baik itu pada fitur node atau edge dalam graf. Penjelasan dianggap stabil jika perbedaan antara penjelasan untuk graf asli dan graf yang telah diberi gangguan (misalnya perubahan kecil pada fitur atau hubungan antar node) tetap minimal. Dengan kata lain, penjelasan model harus tidak terlalu sensitif terhadap perubahan kecil pada data input.

Untuk mengukur stabilitas penjelasan graf (GES), akan menggunakan jarak kosinus (cosine distance) antara dua penjelasan yang dihasilkan. Penjelasan pertama diambil dari graf asli, sedangkan penjelasan kedua diambil setelah graf mengalami perubahan kecil, yang diwakili dengan subgraf yang diberi gangguan. Jarak kosinus mengukur seberapa besar perbedaan arah antara dua vektor, dalam hal ini antara penjelasan graf asli dan yang telah terganggu. Penurunan dalam stabilitas penjelasan mengindikasikan bahwa model cenderung membuat keputusan yang sangat bergantung pada data input yang sensitif terhadap perubahan kecil. Rumus untuk Graph Explanation Stability (GES) adalah sebagai berikut.

$$GES(M_{S_u}^p, M_{S_{u'}}^p) = \max D(M_{S_u}^p, M_{S_{u'}}^p), \forall S_{u'} \in \beta(S_u) \dots\dots\dots(6.3)$$

Di mana:

- $D(M_{S_u}^p, M_{S_{u'}}^p)$ adalah jarak kosinus antara penjelasan yang dihasilkan oleh model untuk subgraf asli (S_u) dan subgraf yang terganggu ($S_{u'}$)
- $\beta(S_u)$ adalah bola radius δ (delta) yang mewakili area sekitar S_u , di mana model menunjukkan perilaku yang serupa meskipun ada gangguan kecil pada data.
- $\max$ menunjukkan nilai maksimum dari jarak kosinus yang diukur pada berbagai perubahan atau gangguan kecil yang diterapkan pada subgraf.

Jika nilai GES mendekati nilai tinggi, ini menunjukkan bahwa penjelasan model tidak stabil, yaitu ada ketergantungan besar pada perubahan kecil dalam input yang menyebabkan perbedaan besar dalam keputusan model. Sebaliknya, nilai GES yang rendah menunjukkan bahwa model memberikan penjelasan yang konsisten meskipun ada perubahan kecil pada data input. Dalam hal ini, penjelasan yang stabil dianggap lebih dapat diandalkan karena menunjukkan bahwa model mengandalkan pola yang lebih kuat dan tidak terlalu dipengaruhi oleh noise atau gangguan kecil.

6.1.4 Graph Explanation Fairness (GECF, GEGF)

Graph Explanation Fairness (GECF) dan Group Explanation Fairness (GEGF) adalah metrik yang digunakan untuk mengevaluasi apakah penjelasan yang diberikan oleh model GNN adil atau tidak bias terhadap kelompok atau atribut tertentu dalam graf. Dalam konteks ini, penjelasan yang adil adalah penjelasan yang mempertimbangkan semua fitur secara seimbang tanpa mempengaruhi hasil keputusan model berdasarkan atribut yang tidak relevan atau protektif.

GECF (Counterfactual Fairness Mismatch) mengukur ketidakcocokan dalam penjelasan antara graf asli dan graf counterfactual, di mana atribut protektif (misalnya, jenis kelamin, ras, atau kategori lainnya) dimodifikasi. Ketika kita memodifikasi atribut tertentu dan memperoleh penjelasan yang sangat berbeda, ini mengindikasikan adanya bias dalam penjelasan model. Dalam rumus, GECF dihitung berdasarkan perbedaan antara penjelasan untuk graf asli dan counterfactual (graf setelah perubahan pada atribut protektif).

$$GECF(M^p, M^s) = D(M^p, M^s) \dots\dots\dots(6.4)$$

Dimana:

- M^p adalah penjelasan yang diprediksi untuk graf asli.
- M^s adalah penjelasan yang diprediksi untuk graf counterfactual, yaitu graf yang telah dimodifikasi dengan mengubah atribut yang dianggap protektif (misalnya, jenis kelamin atau ras).
- $D(M^p, M^s)$ adalah ukuran jarak antara penjelasan graf asli dan penjelasan graf counterfactual.

Semakin tinggi nilai GECF, semakin besar ketidakadilan atau bias yang ada dalam penjelasan. Sebaliknya, jika perubahan pada atribut protektif tidak mempengaruhi penjelasan secara signifikan, GECF akan memiliki nilai yang lebih rendah, menunjukkan bahwa model cenderung memberikan penjelasan yang lebih adil dan tidak bias.

Sedangkan GEGF (Group Explanation Fairness Mismatch) mengukur ketidakadilan pada tingkat kelompok, bukan hanya pada tingkat individu. Metrik

ini menghitung perbedaan penjelasan antara graf asli dan graf yang diubah, dengan menilai apakah perubahan pada fitur-fitur penting yang diidentifikasi oleh penjelasan mempengaruhi hasil keputusan model menggunakan rumus berikut.

$$GEF(\hat{y}_K, \hat{y}_K^{Eu}) = |SP(\hat{y}_K) - SP(\hat{y}_K^{Eu})| \dots\dots\dots(6.5)$$

Dimana:

- $\hat{y}_K$ adalah prediksi yang dihasilkan untuk kumpulan graf asli menggunakan fitur yang dianggap penting oleh penjelasan model.
- $\hat{y}_K^{Eu}$ adalah prediksi yang dihasilkan untuk kumpulan graf setelah melakukan modifikasi pada fitur yang dianggap penting (berdasarkan penjelasan model).
- SP adalah Statistical Parity (SP) mengukur apakah prediksi untuk dua kelompok (misalnya, dua kategori yang berbeda dalam graf) memiliki distribusi hasil yang setara.

Jika hasil dari prediksi pada graf asli dan graf yang diubah sangat berbeda (ditunjukkan oleh $|SP(\hat{y}_K) - SP(\hat{y}_K^{Eu})|$), maka model dianggap tidak adil atau tidak mempertahankan keseimbangan antara kelompok-kelompok tersebut. Jika nilai GEF tinggi, menunjukkan bahwa model tidak mempertahankan keseimbangan atau keadilan antar kelompok. Dalam hal ini, penjelasan model menunjukkan ketidakadilan, di mana beberapa kelompok mungkin diberi perhatian lebih.

Secara keseluruhan, baik GECF maupun GEF adalah metrik yang mengukur bias dalam penjelasan yang diberikan oleh model GNN terhadap atribut yang dilindungi atau kelompok tertentu. Dengan metrik ini, dapat digunakan untuk mengetahui apakah model tersebut memberikan penjelasan yang adil atau tidak.

6.2 Analisis Source Code Metrik Evaluasi

Pada subbab ini, akan dibahas secara rinci implementasi kode dari beberapa metrik evaluasi yang digunakan dalam framework GraphXAI untuk mengukur kualitas penjelasan model Graph Neural Network (GNN). Metrik-metrik tersebut

meliputi Graph Explanation Accuracy (GEA), Graph Explanation Faithfulness (GEF), Graph Explanation Stability (GES), serta metrik fairness seperti Graph Explanation Counterfactual Fairness (GECF) dan Group Explanation Fairness (GEGF).

Pembahasan fokus pada bagaimana masing-masing metrik dihitung secara teknis menggunakan kode program, serta bagaimana metrik tersebut mengukur aspek penting dari penjelasan seperti kesesuaian dengan ground truth, kesetiaan terhadap proses prediksi model, kestabilan penjelasan terhadap gangguan data, dan keadilan dalam penjelasan. Dengan pemahaman yang mendalam terhadap kode ini, diharapkan dapat memperkuat keandalan evaluasi explainability pada GNN dan memberikan gambaran yang jelas mengenai proses evaluasi yang digunakan dalam penelitian ini.

6.2.1 Analisis Source Code Graph Explanation Accuracy (GEA)

Fungsi `graph_exp_acc` digunakan untuk menghitung Graph Explanation Accuracy (GEA), yang mengukur sejauh mana penjelasan yang dihasilkan oleh explainer sesuai atau akurat dengan penjelasan ground truth yang ada. Berikut penjelasan rinci dari setiap baris kode dalam fungsi ini:

Segmen Program 6.1 Fungsi untuk mendapatkan nilai akurasi explainer

```

1: def graph_exp_acc(gt_exp: Explanation, generated_exp:
2:   Explanation, threshold = 0.8) -> float:
3:     EPS = 1e-09
4:     thresh = threshold
5:     relative_positives = (gt_exp.node_imp ==
6:       1).nonzero(as_tuple=True)[0]
7:     true_nodes = [gt_exp.enc_subgraph.nodes[i].item() for i
8:       in relative_positives]
9:
10:    # Accessing the enclosing subgraph. Will be the same for
11:    both explanation.:
12:    exp_subgraph = generated_exp.enc_subgraph
13:
14:    calc_node_imp = generated_exp.node_imp
15:
16:    TPs = []
17:    FPs = []
18:    FNs = []
19:    for i, node in enumerate(exp_subgraph.nodes):
20:        # Restore original node numbering

```

Segmen Program 6.1 (Lanjutan)

```

21:         positive = calc_node_imp[i].item() > thresh
22:         if positive:
23:             if node in true_nodes:
24:                 TPs.append(node)
25:             else:
26:                 FPs.append(node)
27:         else:
28:             if node in true_nodes:
29:                 FNs.append(node)
30:         TP = len(TPs)
31:         FP = len(FPs)
32:         FN = len(FNs)
33:         JAC = TP / (TP + FP + FN + EPS)
34:         # print(f'TP / (TP+FP+FN) edge score of gnn explainer is
35:         {edge_score}')
36:         return JAC

```

Fungsi ini dimulai dengan mendefinisikan tiga argumen yang diperlukan: `gt_exp` (penjelasan ground truth), `generated_exp` (penjelasan yang dihasilkan oleh explainer), dan `threshold` (ambang batas yang digunakan untuk menentukan apakah sebuah node dianggap penting atau tidak, dengan nilai default 0.8) (baris 1-2). Fungsi ini pertama-tama menginisialisasi nilai kecil `EPS` untuk menghindari pembagian dengan nol pada perhitungan selanjutnya, serta menyimpan ambang batas dalam variabel `thresh` (baris 3). Selanjutnya, fungsi ini menentukan node-node yang dianggap penting dalam penjelasan ground truth dengan mencari nilai `node_imp` yang bernilai 1, yang menunjukkan node tersebut dianggap penting. Indeks dari node yang penting tersebut kemudian digunakan untuk mengakses node-node yang relevan dari subgraf dalam penjelasan ground truth dan menyimpannya dalam daftar `true_nodes` (baris 5-8).

Setelah itu, fungsi ini mengakses subgraf yang dihasilkan oleh explainer melalui `generated_exp.enc_subgraph` dan mengidentifikasi nilai `node_imp` pada penjelasan yang dihasilkan. Kemudian, tiga daftar kosong (`TPs`, `FPs`, dan `FNs`) dibuat untuk menyimpan node-node yang dikategorikan sebagai True Positives (TP), False Positives (FP), dan False Negatives (FN) (baris 10-18).

Proses perhitungan TP, FP, dan FN dilakukan dengan mengiterasi setiap node dalam subgraf yang dihasilkan oleh explainer. Jika sebuah node dianggap penting oleh explainer (nilai `node_imp` lebih besar dari ambang batas), maka fungsi akan memeriksa apakah node tersebut juga terdapat dalam daftar `true_nodes`. Jika

ya, node tersebut dianggap sebagai True Positive; jika tidak, dianggap sebagai False Positive. Sebaliknya, jika node tidak dianggap penting oleh explainer namun seharusnya penting menurut ground truth, maka node tersebut dikategorikan sebagai False Negative (baris 19-32).

Setelah semua node diperiksa, jumlah TP, FP, dan FN dihitung. Graph Explanation Accuracy (GEA) dihitung menggunakan Jaccard Index dengan rumus:

$$JAC = \frac{TP}{TP+FP+FN+EPS} \dots\dots\dots(6.6)$$

Indeks Jaccard ini memberikan nilai yang mengukur kesamaan antara dua set, dalam hal ini penjelasan ground truth dan penjelasan yang dihasilkan. Semakin tinggi nilai Jaccard Index, semakin baik penjelasan yang dihasilkan sesuai dengan penjelasan ground truth. Akhirnya, fungsi ini mengembalikan nilai Jaccard Index sebagai hasil evaluasi Graph Explanation Accuracy (baris 37).

6.2.2 Analisis Source Code Graph Explanation Faithfulness (GEF)

Fungsi `graph_exp_faith` digunakan untuk menghitung Graph Explanation Faithfulness (GEF), yang merupakan ukuran seberapa setia penjelasan yang dihasilkan oleh model terhadap keputusan yang diambil oleh model tersebut. GEF dihitung dengan membandingkan distribusi probabilitas antara dua kondisi graf: graf asli dan graf yang telah dimodifikasi untuk melihat pengaruh perubahan pada node, fitur, atau edge. Source code dapat dilihat pada Lampiran Segmen Program A.8.

Fungsi ini dimulai dengan memeriksa apakah informasi `feature_imp` (feature importance) tersedia dalam penjelasan yang dihasilkan oleh explainer (baris 5). Jika tersedia, fungsi ini pertama-tama mengidentifikasi fitur-fitur `top_k` terpenting berdasarkan nilai `feature_imp` (baris 8-10). Pemetaan node kemudian dilakukan dengan mencari `node_map` untuk `node_id` yang relevan dalam penjelasan, menggunakan referensi node yang ada di `generated_exp.node_reference` (baris 12-13). Selanjutnya, fungsi ini menghitung

softmax untuk graf asli dengan menjalankan model pada `pert_x` (salinan dari fitur node (X)) dan `edge_index` (EIDX) yang asli (baris 15-18). Hasilnya disimpan dalam `org_softmax`, yang menunjukkan distribusi probabilitas prediksi untuk `node_id` (baris 19).

Fungsi kemudian memodifikasi graf untuk menghasilkan graf terperturbasi (baris 22). Fitur node yang tidak penting (selain yang ada dalam `top_k_features`) diubah dengan menambahkan noise Gaussian pada fitur yang relevan menggunakan `torch.normal()`, yang menghasilkan `pert_x` yang telah dimodifikasi (baris 26-31). `pert_vec` dihitung dengan menerapkan model pada graf yang terperturbasi dan mendapatkan prediksi untuk `node_id` yang sama (baris 32). Softmax kemudian diterapkan pada `pert_vec` untuk menghitung distribusi probabilitas untuk graf yang terperturbasi, yang disimpan dalam `pert_softmax` (baris 35).

Setelah itu, fungsi ini memeriksa apakah `node_imp` (node importance) tersedia (baris 37). Jika ada, `top_k_nodes` dihitung untuk memilih node terpenting berdasarkan nilai `node_imp`. Fungsi ini kemudian membuat daftar `rem_nodes`, yaitu node yang tidak termasuk dalam `top_k_nodes`, dan melakukan masking pada node-node tersebut dalam graf yang terperturbasi (baris 44-49). Setelah itu, `org_vec` akan diisi dengan prediksi untuk seluruh node dalam graf. Akan diterapkan fungsi softmax pada prediksi model untuk menghasilkan distribusi probabilitas di mana setiap elemen mewakili probabilitas kelas untuk node yang dianalisis. Softmax di sini digunakan untuk mengubah nilai logit menjadi probabilitas (baris 55).

Selanjutnya, akan dihitung softmax dari graf yang diperturbasi dan melakukan masking pada node tidak penting (baris 58-65). Jika `edge_imp` (edge importance) tersedia, fungsi menghitung `top_k_edges` untuk memilih edge terpenting dan memodifikasi edge index pada graf yang terperturbasi untuk hanya mempertahankan edge yang relevan (baris 70-75). Fungsi kemudian menghitung prediksi untuk graf yang terperturbasi dengan edge yang telah dimodifikasi. Akhirnya, fungsi menghitung Kullback-Leibler (KL) Divergence antara distribusi softmax untuk graf asli dan graf yang terperturbasi, yang memberikan ukuran Graph Explanation Faithfulness (GEF). Semakin kecil perbedaan antara distribusi probabilitas kedua graf, semakin faithful penjelasan tersebut. Fungsi ini

mengembalikan nilai GEF, yang menunjukkan seberapa stabil penjelasan yang diberikan oleh model meskipun ada gangguan kecil pada graf. Penjelasan yang faithful adalah penjelasan yang tetap konsisten meskipun ada perubahan kecil pada graf, menunjukkan bahwa model memberikan penjelasan yang dapat dipercaya dan konsisten

6.2.3 Analisis Source Code Graph Explanation Stability (GES)

Fungsi `graph_exp_stability` digunakan untuk mengukur stabilitas penjelasan model dengan membandingkan penjelasan yang dihasilkan untuk graf asli dan graf yang terperturbasi. Penjelasan yang stabil berarti hasil penjelasan tidak berubah secara signifikan meskipun ada perubahan kecil pada graf, seperti gangguan pada fitur node atau koneksi edge. Fungsi ini mengulang eksperimen beberapa kali, dan pada setiap eksperimen, graf akan dimodifikasi dan penjelasan akan dihitung ulang untuk mengevaluasi konsistensinya. Source Code untuk fungsi ini dapat dilihat pada Lampiran Segmen Program A.9.

Pada awal fungsi, variabel GES diinisialisasi sebagai list kosong, yang nantinya akan menyimpan nilai Graph Explanation Stability (GES) untuk setiap eksperimen (baris 4). Fungsi juga menentukan jumlah eksperimen yang akan dilakukan dengan `num_run = 25`, yang berarti eksperimen ini akan diulang sebanyak 25 kali. `for run in range(num_run)` menjalankan eksperimen berulang kali (baris 5-6).

Setiap eksperimen dimulai dengan perturbasi graf, yang dilakukan dengan memodifikasi koneksi antar node menggunakan `rewire_edges()` (baris 10-12). Kemudian, fitur node untuk `node_id` yang diberikan juga dimodifikasi dengan menambahkan noise Gaussian menggunakan `torch.normal()` (baris 17-18). Hal ini bertujuan untuk melihat apakah perubahan kecil pada graf dapat mempengaruhi penjelasan yang dihasilkan oleh model.

Setelah gangguan diterapkan, fungsi `check_delta()` digunakan untuk memeriksa apakah perubahan pada graf mempengaruhi hasil prediksi model secara signifikan. Jika perubahan tersebut cukup signifikan, penjelasan dihitung menggunakan CAM (Class Activation Mapping) yang digunakan untuk

menghasilkan penjelasan berbasis aktivasi. CAM akan memberikan penjelasan tentang node yang terperturbasi dengan menunjukkan fitur atau bagian dari graf yang berkontribusi paling besar pada keputusan model. Pada baris 28, `cam.get_explanation_node()` akan menghasilkan penjelasan untuk node yang terperturbasi berdasarkan fitur dan edge pada graf yang terperturbasi. Lalu penjelasan `cam_pert_exp.node_imp` dinormalisasi ke dalam rentang $[0, 1]$ dengan membaginya dengan nilai maksimum di antara semua node (baris 35-38).

Penjelasan yang dihasilkan kemudian dibandingkan dengan penjelasan yang sudah ada. Jika `feature_imp` tersedia, fitur yang paling relevan dipilih dengan menggunakan `top_k`, dan mask dibuat untuk fitur yang paling penting dalam penjelasan asli dan yang terperturbasi. Perbedaan antara penjelasan asli dan terperturbasi dihitung menggunakan cosine similarity (baris 42-57). Proses yang serupa dilakukan untuk `node_imp` dan `edge_imp`, yang mengukur pentingnya node dan edge dalam penjelasan. Jika informasi `node_imp` atau `edge_imp` tersedia, penjelasan dihitung dengan memodifikasi node atau edge yang tidak relevan dengan masking, dan perbedaan antara penjelasan yang asli dan yang terperturbasi dihitung lagi dengan cosine similarity (baris 59-98).

Setelah eksperimen selesai, nilai GES dihitung dan ditambahkan ke array GES. Fungsi ini kemudian mengembalikan nilai maksimum dari GES, yang menunjukkan seberapa stabil penjelasan yang dihasilkan oleh model (baris 102). Nilai GES yang lebih tinggi menunjukkan penjelasan yang lebih stabil, di mana perubahan kecil pada graf tidak menyebabkan perubahan besar dalam penjelasan yang dihasilkan oleh model. Sebaliknya, nilai GES yang rendah menunjukkan bahwa penjelasan sangat dipengaruhi oleh gangguan yang diterapkan, yang mungkin menandakan ketidakstabilan atau bias dalam penjelasan model.

6.2.4 Analisis Source Code Graph Explanation Fairness (GECF)

Fungsi `graph_exp_cf_fairness` digunakan untuk menghitung Graph Explanation Counterfactual Fairness (GECF), yang mengukur ketidakcocokan penjelasan antara graf asli dan graf counterfactual (graf yang dimodifikasi dengan mengubah atribut protektif seperti jenis kelamin atau ras). Source Code dapat

dilihat pada Lampiran Segmen Program A.10. Fungsi ini dimulai dengan menginisialisasi variabel `GECF_feat`, `GECF_node`, dan `GECF_edge` dengan nilai `None`, yang nantinya akan diisi dengan hasil perhitungan GECF untuk fitur, node, dan edge (baris 5-7). Kemudian, fungsi memeriksa apakah data yang diperlukan sudah ada, jika tidak, ia akan menggunakan `shape_graph.get_graph()` untuk mendapatkan data graf. Fitur node (X), label (Y), dan edge index (EIDX) dipindahkan ke device yang ditentukan (baris 9-13).

Setelah data siap, salinan dari fitur node (`pert_x`) dibuat untuk melakukan perturbasi. Fungsi ini kemudian memodifikasi atribut sensitif node yang relevan (`node_id` dan `sens_idx`) dengan mengubahnya menjadi nilai yang berbeda (menggunakan `1 - pert_x[node_id, sens_idx]`) (baris 16-17). Perturbasi ini dimaksudkan untuk menguji counterfactual fairness dengan memodifikasi atribut protektif. Fungsi `check_delta` digunakan untuk memastikan apakah perubahan tersebut signifikan, yang berarti perbedaan dalam penjelasan model setelah graf dimodifikasi (baris 21-23).

Jika perubahan tersebut signifikan, fungsi melanjutkan untuk menghasilkan penjelasan Counterfactual dengan menggunakan GNN Explainer. Penjelasan ini dihitung dengan `gnnexpr.get_explanation_node`, yang menghasilkan penjelasan untuk node yang terperturbasi (baris 27-30). Selanjutnya, penjelasan yang dihasilkan dibandingkan dengan penjelasan yang sudah ada, dimulai dengan `feature_imp`. Jika `feature_imp` tersedia, fitur terpenting dipilih berdasarkan nilai `top_k`, dan mask dibuat untuk fitur yang paling relevan pada penjelasan asli dan yang terperturbasi (baris 33-41). Cosine similarity digunakan untuk mengukur perbedaan antara penjelasan asli dan yang terperturbasi pada fitur node, dan nilai GES untuk fitur dihitung (baris 45-47).

Jika `node_imp` juga tersedia, fungsi ini melanjutkan untuk membandingkan node importance pada penjelasan asli dan terperturbasi. `top_k_nodes` dihitung untuk memilih node terpenting berdasarkan `node_imp` (baris 50-51). Daftar `all_nodes` kemudian dibuat dengan mengambil irisan antara node yang relevan dari penjelasan asli dan yang terperturbasi (baris 52-54). Penjelasan asli dan terperturbasi untuk node tersebut dibandingkan menggunakan cosine similarity

(baris 70-73). Perhitungan yang serupa dilakukan untuk edge importance jika edge_imp tersedia, di mana perbedaan antara penjelasan edge asli dan terperturbasi dihitung dengan cara yang sama (baris 93-130).

Akhirnya, akan dikembalikan tiga nilai GECF: untuk fitur, node, dan edge (baris 132). Ini menunjukkan sejauh mana penjelasan yang dihasilkan oleh model adil atau tidak bias terhadap atribut protektif yang diubah.

6.2.5 Analisis Source Code Group Explanation Fairness (GEGF)

Fungsi graph_exp_group_fairness ini digunakan untuk mengukur Group Explanation Fairness (GEGF), yang mengidentifikasi apakah penjelasan yang dihasilkan oleh model GNN adil atau tidak terhadap kelompok-kelompok tertentu dalam graf, terutama dalam hal fitur, node, dan edge. Source Code dapat dilihat pada Lampiran Segmen Program A.11. Fungsi ini dimulai dengan menginisialisasi beberapa variabel seperti GGF_feat, GGF_node, dan GGF_edge, yang nantinya akan menyimpan nilai GEGF untuk fitur, node, dan edge (baris 15-18). Fungsi ini kemudian memeriksa apakah data graf sudah ada, jika tidak, data graf diambil menggunakan shape_graph.get_graph() (baris 19-20). Fitur node X, label Y, dan edge index EIDX kemudian dipindahkan ke perangkat yang ditentukan (misalnya, CPU atau GPU) (baris 21-23).

Selanjutnya, fungsi memeriksa apakah graf yang sudah di-cache (G) ada (baris 25). Jika tidak, graf diubah menjadi format NetworkX menggunakan to_networkx_conv(), yang digunakan untuk mengubah struktur graf sehingga bisa digunakan dalam rewiring_edges (baris 26). Kemudian, data_for_rewire disiapkan untuk digunakan dalam perubahan struktur graf (baris 28). Fungsi ini juga menentukan jumlah sampel tetangga yang akan digunakan dalam eksperimen (jumlah num_samples) (baris 31).

Fungsi kemudian melakukan loop sebanyak num_samples untuk mengevaluasi keadilan penjelasan. Pada iterasi pertama, prediksi dihitung berdasarkan graf asli tanpa modifikasi, menggunakan softmax pada output model untuk node yang dianalisis (node_id) (baris 34-39). Pada iterasi berikutnya, perturbasi diterapkan pada fitur node yang relevan dengan menggunakan

`perturb_node_features()`, yang mengubah nilai fitur tertentu untuk node yang dianalisis (baris 41-58). Gangguan juga diterapkan pada edge index dengan menggunakan `rewire_edges()`, yang mengubah koneksi antar node untuk menguji apakah perubahan pada graf memengaruhi penjelasan secara signifikan (baris 49-52) .

Setelah gangguan diterapkan, fungsi `check_delta` digunakan untuk memverifikasi apakah perubahan tersebut signifikan, yang berarti mempengaruhi keputusan model (baris 62-64). Jika perubahan tersebut dianggap signifikan, fungsi melanjutkan untuk menghitung penjelasan menggunakan GNN Explainer dengan `gnnexpr.get_explanation_node` untuk node yang terpengaruh. Penjelasan yang dihasilkan kemudian dibandingkan dengan penjelasan yang sudah ada. Jika `feature_imp` tersedia, masker untuk fitur terpenting dihitung berdasarkan nilai `top_k`. `ori_exp_mask` dan `pert_exp_mask` dibuat untuk mencocokkan fitur yang relevan dalam penjelasan asli dan yang terpengaruh, lalu akan disimpan ke dalam array prediksi explainer (baris 66-84) .

Fungsi juga memeriksa apakah `node_imp` (node importance) tersedia. Jika ada, `top_k_nodes` dihitung untuk memilih node terpenting berdasarkan nilai `node_imp`. Node yang tidak penting dihapus dengan masking dan dihitung prediksi untuk graf yang telah dimodifikasi (baris 86-114). Proses serupa dilakukan untuk `edge_imp` (edge importance) jika tersedia, di mana edge yang tidak penting dimodifikasi atau dihapus, dan prediksi dihitung untuk graf yang telah terganggu (baris 116-158).

Setelah semua eksperimen selesai, Statistical Parity (SP) dihitung untuk fitur, node, dan edge, yang mengukur apakah perubahan pada fitur atau struktur graf mempengaruhi prediksi secara tidak adil antara kelompok yang berbeda. Fungsi ini mengembalikan nilai GEGF untuk fitur, node, dan edge, yang menunjukkan seberapa adil model dalam memberikan penjelasan untuk kelompok-kelompok yang ada dalam graf berdasarkan atribut yang sensitif (baris 163-173). Jika nilai GEGF tinggi, itu menunjukkan bahwa penjelasan model mungkin tidak adil dan ada bias yang signifikan antara kelompok yang berbeda.

6.3 Pengujian dan Evaluasi dengan Dataset Sintetis ShapeGGen

Pada subbab ini, dilakukan evaluasi praktis terhadap metode explainability yang diterapkan pada model GNN dengan menggunakan dataset sintetis ShapeGGen. Hasil evaluasi diukur dengan métrik-métrik evaluasi seperti Graph Explanation Accuracy (GEA) dan Graph Explanation Faithfulness (GEF), yang berfungsi untuk menilai seberapa baik penjelasan yang diberikan oleh masing-masing metode explainability.

6.3.1 Evaluasi Graph Explanation Accuracy (GEA)

Untuk mengevaluasi Graph Explanation Accuracy (GEA), kita menggunakan fungsi `graph_exp_acc` untuk menghitung akurasi penjelasan yang dihasilkan oleh GradExplainer, GNNExplainer, PGMEExplainer, dan GraphLIME dibandingkan dengan ground-truth explanation yang sudah diketahui dari dataset sintetis ShapeGGen. Berikut adalah kode untuk menghitung GEA pada setiap explainer

Segmen Program 6.2 Kode Implementasi Graph Explanation Accuracy

```
1: gea_grad = graph_exp_acc(ground_truth, grad_explanation)
2: gea_gnn = graph_exp_acc(ground_truth, gnn_explanation)
3: gea_pgm = graph_exp_acc(ground_truth, pgm_explanation)
4: gea_lime = graph_exp_acc(ground_truth, lime_exp)
5:
6: print("Graph Explanation Accuracy:")
7: print(f"Grad: {gea_grad:.4f}")
8: print(f"GNNExplainer: {gea_gnn:.4f}")
9: print(f"PGMEExplainer: {gea_pgm:.4f}")
10: print(f"GraphLIME: {gea_lime:.4f}")
```

Setelah kode di atas diimplementasikan, dihasilkan evaluasi Graph Explanation Accuracy yang menunjukkan:

- GradExplainer dan PGMEExplainer memiliki nilai 0.0000, yang menandakan bahwa penjelasan yang diberikan sangat berbeda dengan ground truth.
- GNNExplainer memiliki nilai 0.8182, yang menunjukkan penjelasan yang cukup akurat dan relevan dengan ground truth.

- GraphLIME memiliki nilai 0.0000, menunjukkan bahwa penjelasan yang diberikan sangat berbeda dari penjelasan yang benar.

6.3.2 Evaluasi Graph Explanation Faithfulness (GEF)

Setelah mengevaluasi GEA, dilanjutkan dengan evaluasi Graph Explanation Faithfulness (GEF), yang mengukur seberapa setia penjelasan yang diberikan terhadap proses pengambilan keputusan model GNN. Fungsi `graph_exp_faith` digunakan untuk menghitung GEF berdasarkan penjelasan yang diberikan oleh masing-masing explainer.

Segmen Program 6.3 Kode Implementasi Graph Explanation Faithfulness

```
1: gef_grad = graph_exp_faith(grad_explanation, dataset, model)
2: gef_gnn = graph_exp_faith(gnn_explanation, dataset, model)
3: gef_pgm = graph_exp_faith(pgm_explanation, dataset, model)
4: gef_lime = graph_exp_faith(lime_exp, dataset, model)
5:
6: print("\nGraph Explanation Faithfulness:")
7: print(f"Grad: {gef_grad:.4f}")
8: print(f"GNNExplainer: {gef_gnn:.4f}")
9: print(f"PGMExplainer: {gef_pgm:.4f}")
10: print(f"GraphLIME: {gef_lime:.4f}")
```

Setelah kode di atas diimplementasikan, dihasilkan evaluasi Graph Explanation Faithfulness yang menunjukkan:

- GradExplainer dan GNNExplainer memiliki nilai GEF yang sangat tinggi yaitu 0.9979 dan 0.9972, yang menunjukkan bahwa penjelasan yang diberikan sangat setia terhadap keputusan model.
- PGMExplainer memiliki nilai GEF 0.9969, juga menunjukkan tingkat kesetiaan yang sangat tinggi.
- GraphLIME memiliki nilai GEF 0.5896, yang lebih rendah, menunjukkan bahwa penjelasan yang diberikan oleh GraphLIME kurang setia terhadap proses pengambilan keputusan model.

6.4 Hasil Evaluasi Pengujian

Dalam penelitian ini, dataset sintetis dihasilkan menggunakan ShapeGGen dengan konfigurasi yang telah ditentukan, seperti jumlah lapisan model, ukuran subgraf, dan probabilitas koneksi antar node. Dataset ini menyediakan graf dengan penjelasan ground-truth yang digunakan untuk menguji metode explainability.

Model GNN yang digunakan adalah Graph Convolutional Network (GCN) yang dilatih pada dataset ShapeGGen dengan menggunakan loss function berbobot berdasarkan distribusi kelas agar model dapat belajar dengan lebih seimbang terhadap kelas minoritas maupun mayoritas. Model ini dilatih selama 300 epoch menggunakan optimizer Adam dengan learning rate dan weight decay yang telah disesuaikan.

Setelah pelatihan, model dievaluasi menggunakan metrik seperti F1 score dan Area Under the ROC Curve (AUROC) untuk memastikan kualitas prediksi model sebelum dilakukan tahap explainability. Hasil evaluasi yang baik pada metrik tersebut menunjukkan bahwa model telah belajar dengan cukup baik, sehingga penjelasan yang dihasilkan oleh metode explainability dapat dipercaya sebagai refleksi dari proses prediksi yang valid.

Hasil evaluasi menggunakan metrik Graph Explanation Accuracy (GEA) dan Graph Explanation Faithfulness (GEF) memberikan gambaran penting mengenai kualitas penjelasan yang dihasilkan oleh masing-masing metode explainability. Nilai GEA yang tinggi mengindikasikan bahwa penjelasan yang diberikan oleh metode tersebut sangat sesuai dengan penjelasan ground-truth, sehingga model mampu menjelaskan keputusan dengan akurat. Sementara itu, nilai GEF yang tinggi menunjukkan bahwa penjelasan tersebut setia terhadap proses pengambilan keputusan model asli, sehingga dapat dipercaya sebagai representasi yang benar dari cara model bekerja. Nilai GES yang baik mengindikasikan stabilitas penjelasan terhadap gangguan kecil pada data input, yang menambah keandalan penjelasan tersebut.

Dari hasil yang diperoleh, GNNExplainer menunjukkan performa yang unggul dengan keseimbangan antara akurasi, kesetiaan, dan stabilitas penjelasan, menjadikannya pilihan yang tepat untuk kasus penggunaan pada dataset sintetis

ShapeGGen. Sebaliknya, meskipun beberapa metode lain seperti GradExplainer dan PGMEExplainer memiliki nilai faithfulness yang baik, nilai akurasinya rendah, yang menunjukkan perlunya perbaikan atau penyesuaian dalam penerapan metode tersebut. GraphLIME menunjukkan performa yang kurang konsisten, sehingga penggunaannya perlu dipertimbangkan dengan hati-hati.

Berdasarkan diskusi ini, pemilihan metode explainability sebaiknya disesuaikan dengan kebutuhan spesifik dan karakteristik data. Metode dengan keseimbangan baik antara akurasi, kesetiaan, dan stabilitas akan memberikan penjelasan yang paling informatif dan dapat diandalkan dalam mendukung interpretasi model GNN.